

Parallel Cart Project — Extended Mapping of the 10 PDF Requirements

Document Info

- Source PDF: parallel-programming-project-2026.pdf (3 pages)
- Extraction date: 2026-05-18
- Project analyzed: parallel-cart (Spring Boot backend)
- Scope: map each non-functional requirement from PDF to concrete code, explain implementation deeply, and provide a before/after validation plan per point.

Executive Summary

This project already implements all ten non-functional requirements listed in the PDF at architecture level. Evidence exists in source code, tests, scripts, and generated reports. Current test status from local run: - Surefire files: 6 - Total tests: 10 - Failures: 0 - Errors: 0 - Skipped: 0

Two important quality notes: 1. Requirement #10 (optimization) is implemented and benchmarked, but the attempted optimization regressed performance. This is still valid engineering because it is measured, documented, and reproducible. 2. Requirement #1 (race-condition proof) has strong implementation and indirect coverage, but a dedicated high-contention checkout concurrency integration test should be added to make evidence stronger.

High-Level Requirement-to-Implementation Map

PDF Point	Requirement	Main Project Implementation
1	Concurrent Access & Data Integrity	Inventory.@Version, retry + jitter, pessimistic fallback in checkout
2	Resource Management & Capacity Control	Bounded thread pools + queue capacities + semaphore fail-fast guard
3	Asynchronous Queues	Transactional outbox + scheduled publisher +

PDF Point	Requirement	Main Project Implementation
4	Batch Processing (Chunks)	Kafka consumers + DLQ Daily sales chunk loop + checkpoint resume + scheduled job
5	Load Distribution	Nginx least_conn with app-1, app-2 and upstream header evidence
6	Distributed Caching	Redis-backed product cache + TTL + post-commit invalidation
7	Concurrency Control (Optimistic/Pessimistic)	Optimistic lock first, pessimistic lock fallback for hot rows
8	Transaction Integrity / ACID	@Transactional checkout all-or-nothing with idempotency
9	Stress Testing ≥ 100 users	k6 stress profile, 120 VUs, zero failed requests in artifact
10	Benchmarking + Bottleneck Analysis	AOP timing + bottleneck hypothesis + before/after numeric comparison

1) Concurrent Access & Data Integrity

Requirement Meaning

System must prevent conflicting writes when many users change shared data (inventory) at same time, and must prove race-condition handling.

What In This Project Implements It

- `src/main/java/com/parallelcart/domain/model/Inventory.java`
 - Uses @Version for optimistic concurrency control.
- `src/main/java/com/parallelcart/service/impl/CartServiceImpl.java`

- Method `reserveInventoryWithRetry(...)` retries optimistic conflicts.
- Method `reserveInventoryWithPessimisticLock(...)` applies DB lock fallback.
- `src/main/java/com/parallelcart/api/controller/GlobalExceptionHandler.java`
 - Maps optimistic conflicts to clear HTTP 409 API response.

Deep Technical Explanation

The race-sensitive shared resource is `inventory.availableQuantity`. During checkout, each cart item attempts reservation. First strategy is optimistic update: read row, subtract quantity, `saveAndFlush`. If another transaction has already changed the same row, JPA throws `ObjectOptimisticLockingFailureException` due to version mismatch.

Instead of failing immediately, code retries up to three attempts with random jitter (20-80 ms). This jitter reduces synchronized retry waves under contention. If retries still collide, flow switches to pessimistic row lock via `findByProductForUpdate (PESSIMISTIC_WRITE)` and retries reservation under exclusive DB lock.

This two-stage strategy gives both performance and correctness: - Low contention: optimistic path is fast and non-blocking. - High contention: pessimistic fallback enforces serialization and protects data integrity.

Concrete Project Evidence

- Optimistic version field: `Inventory.java` line with `@Version`.
- Retry loop and jitter: `CartServiceImpl.reserveInventoryWithRetry`.
- Fallback lock path: `CartServiceImpl.reserveInventoryWithPessimisticLock`.
- API conflict response path: `GlobalExceptionHandler` optimistic lock handler.

Before/After Test Plan (Point 1)

Goal

Show race condition behavior before protection and correctness after protection.

Before (Bad State) Test Design

- Create branch `exp/p1-before-unsafe-inventory`.

- In branch, temporarily bypass optimistic version + retry/fallback path.
 - Example: remove retry logic and direct decrement/update.
- Run parallel checkout load on same product (single SKU hotspot).

Expected bad outcomes: - Oversell or inconsistent inventory counts. - More random conflicts or data corruption possibilities. - Potential mismatch between successful checkouts and inventory decrements.

After (Good State) Test Design

- Switch back to current branch implementation.
- Run same parallel hotspot scenario.

Expected good outcomes: - No oversell. - Deterministic inventory floor checks. - Conflicts handled as controlled retries/409 responses rather than corruption.

Suggested Execution Commands

- Functional integrity baseline:
 - `./mvnw -q -Dtest=CheckoutTransactionIntegrityTest test`
- Hotspot runtime scenario (manual stress on same product):
 - repeated concurrent `POST /api/carts/{userId}/checkout` requests with different idempotency keys.
- Add verification SQL checks:
 - compare initial inventory, successful paid orders, and final inventory.

Metrics to Capture

- Inventory mismatch count.
 - Number of optimistic conflicts.
 - Number of successful orders with valid stock.
 - Any negative stock incidents (must be zero after fix).
-

2) Resource Management & Capacity Control

Requirement Meaning

System must control parallel work volume to avoid collapse, while not over-throttling.

What In This Project Implements It

- `src/main/java/com/parallelcart/config/ResourceManagementConfig.java`
 - Bounded async thread-pool and scheduler pool.
 - Queue capacity set explicitly.
 - Rejection policy `CallerRunsPolicy` for backpressure propagation.
- `src/main/resources/application-local.yml`
 - Configurable limits for pools and queue sizes.
- `src/main/java/com/parallelcart/service/impl/SemaphoreCheckoutSaturationGuard.java`
 - Limits concurrent checkouts with semaphore.
- `src/main/java/com/parallelcart/api/controller/CartController.java`
 - Checkout path is wrapped by saturation guard.
- `src/main/java/com/parallelcart/api/controller/GlobalExceptionHandler.java`
 - Saturation returns HTTP 503 (`SystemSaturated`).

Deep Technical Explanation

Capacity protection exists at two layers: 1. Execution resources: bounded pool sizes and queue prevent runaway thread creation and unbounded memory growth. 2. Business hotspot protection: semaphore caps number of simultaneous checkout executions. When full, request fails fast with 503.

This is better than allowing infinite queueing because it protects tail latency and process health. Caller receives explicit overload signal and can retry with backoff.

Concrete Project Evidence

- Pool + queue config and bounded handler in `ResourceManagementConfig`.
- Threshold value in `application-local.yml` (`app.backpressure.checkout.max-concurrent`).
- Guard integration in controller checkout endpoint.
- Saturation API behavior validated in `CartControllerSaturationTest`.

Before/After Test Plan (Point 2)

Goal

Demonstrate uncontrolled saturation before capacity controls, and graceful degradation after controls.

Before (Bad State) Test Design

- Disable guard (or use no-op guard under non-test profile for experiment).
- Increase traffic with concurrent checkouts until queue and threads saturate.

Expected bad outcomes: - Rising p95/p99 latency. - Potential timeouts and thread starvation. - Less predictable recovery.

After (Good State) Test Design

- Enable current semaphore guard and bounded pools.
- Run same stress profile.

Expected good outcomes: - Controlled 503 at saturation point. - Stable process, no crash. - Better tail-latency stability for accepted traffic.

Suggested Execution Commands

- `./mvnw -Dtest=CartControllerSaturationTest test`
- `./scripts/verify_phase4.sh t3`
- `./scripts/verify_phase4.sh t4`

Metrics to Capture

- HTTP code distribution (200 vs 503).
 - Throughput under overload.
 - p95 and max latency.
 - App health endpoint continuity.
-

3) Asynchronous Queues

Requirement Meaning

Non-immediate tasks (invoice, notification) must leave request critical path.

What In This Project Implements It

- Outbox write during checkout transaction:
 - `src/main/java/com/parallelcart/infra/messaging/OutboxEventService.java`
- Scheduled outbox publisher:
 - `src/main/java/com/parallelcart/infra/messaging/OutboxPublisherJob.java`
- Kafka publisher:

- `src/main/java/com/parallelcart/infra/messaging/KafkaOrderEventPublisher.java`
- Consumers:
 - `InvoiceOrderCreatedConsumer`
 - `NotificationOrderCreatedConsumer`
- Reliability/error handling:
 - `KafkaConsumerErrorHandlingConfig` (retry + DLQ)

Deep Technical Explanation

Checkout writes business data and outbox event in same transaction. This avoids dual-write inconsistency (DB success + event failure split). Later, scheduled publisher loads pending outbox records and publishes them to Kafka. Status transitions (PENDING -> PUBLISHED or FAILED) are persisted with attempt counters.

Consumers are idempotent (existence check by order ID before insert), so at-least-once delivery does not duplicate invoice/notification rows.

This architecture shortens checkout path and increases reliability under transient broker/consumer issues.

Concrete Project Evidence

- Outbox status model and publish attempts in `OutboxEvent`.
- Scheduled publisher every configured delay in `OutboxPublisherJob`.
- Consumer group separation (invoice-workers, notification-workers).
- Scripted verification in `scripts/verify_phase.sh p3`.

Before/After Test Plan (Point 3)

Goal

Show checkout responsiveness and consistency improvement when async queues used instead of inline side effects.

Before (Bad State) Test Design

- Experimental branch with invoice + notification performed inline inside checkout transaction.
- Inject artificial delay (e.g., 150-300 ms) into side-effect operations.

Expected bad outcomes: - Higher checkout latency. - Larger failure blast radius (side-effect error may fail user checkout).

After (Good State) Test Design

- Restore outbox + Kafka async pipeline.
- Same load profile.

Expected good outcomes: - Lower checkout latency. - Successful checkout still possible while async jobs retry. - Clear observability of async progress via outbox and consumer records.

Suggested Execution Commands

- `./scripts/verify_phase.sh p3`
- Check DB rows for `outbox_events`, `invoices`, `notification_logs` before/after.

Metrics to Capture

- Checkout API latency.
 - Outbox publish lag and attempts.
 - Side-effect completion lag (event time to invoice/notification row time).
 - Failure recovery count via retry/DLQ logs.
-

4) Batch Processing (Chunks)

Requirement Meaning

Large daily sales aggregation must run in chunked background processing, not giant single in-memory pass.

What In This Project Implements It

- Service logic:
 - `src/main/java/com/parallelcart/service/impl/DailySalesAggregationServiceImpl.java`
- Scheduled job trigger:
 - `src/main/java/com/parallelcart/infra/batch/DailySalesAggregationJob.java`
- Checkpoint state:
 - `src/main/java/com/parallelcart/domain/model/DailySalesCheckpoint.java`
- Summary output table model:
 - `DailySalesSummary`
- Tunable config:

- `application-local.yml` (`chunk-size`, `max-chunks-per-run`)

Deep Technical Explanation

Process flow: 1. Build date window for previous day. 2. Load or create checkpoint. 3. Repeatedly fetch paid orders in ascending ID chunks (PageRequest limited by `chunk-size`). 4. Update running totals in checkpoint (`paidOrderCount`, `totalRevenue`, `processedChunks`, `lastProcessedOrderId`). 5. If no more rows, write final summary and mark checkpoint COMPLETED. 6. If `max-chunks-per-run` set, process can stop early and resume next run.

This design protects memory, allows resumability, and supports huge datasets.

Concrete Project Evidence

- Chunk query by status/date/id and pageable in `OrderRepository` + `DailySalesAggregationServiceImpl`.
- Resume semantics verified by `DailySalesAggregationCheckpointResumeTest`.
- Aggregation correctness validated by `DailySalesAggregationServiceTest`.

Before/After Test Plan (Point 4)

Goal

Show chunking/resume reliability vs non-chunked monolithic batch behavior.

Before (Bad State) Test Design

- Experimental branch with one-shot full-day fetch (no chunking/checkpoint).
- Load large synthetic paid orders.

Expected bad outcomes: - Higher memory pressure. - Restart causes full rework. - Poor fault tolerance mid-run.

After (Good State) Test Design

- Use current chunk + checkpoint implementation.
- Force partial runs with `max-chunks-per-run=1`.

Expected good outcomes: - Stable memory profile. - Resume from last processed order. - Accurate final summary after multiple runs.

Suggested Execution Commands

- `./mvnw -Dtest=DailySalesAggregationServiceTest test`

- `./mvnw -Dtest=DailySalesAggregationCheckpointResumeTest test`
- `./scripts/verify_phase5.sh t1`
- `./scripts/verify_phase5.sh t2`

Metrics to Capture

- Peak memory during job.
 - Number of chunks processed.
 - Resume correctness after interruption.
 - Total runtime for equivalent dataset.
-

5) Load Distribution

Requirement Meaning

Requests must be distributed across multiple app instances with justified strategy.

What In This Project Implements It

- Multi-instance topology in `docker-compose.yml`:
 - `app-1`, `app-2`, and `nginx` profile.
- Nginx strategy in `docker/nginx/nginx.conf`:
 - `least_conn` load-balancing algorithm.
 - `X-Upstream-Addr` response header for visibility.
- Strategy doc:
 - `docs/load-distribution-strategy.md`.
- Verification automation:
 - `scripts/verify_phase5.sh t3` and `t4`.

Deep Technical Explanation

Nginx fronts two identical Spring Boot instances and routes each incoming request to backend with lowest active connection count (`least_conn`). Compared with pure round-robin, this handles mixed request durations better by avoiding overloaded instance stickiness under long-running operations.

Header `X-Upstream-Addr` exposes selected backend per request; sampling over burst traffic proves distribution is occurring.

Concrete Project Evidence

- Upstream pool + `least_conn`: nginx config.

- Two app services defined under lb profile in compose.
- Validation approach and tradeoffs documented in load strategy doc.

Before/After Test Plan (Point 5)

Goal

Show single-instance bottleneck before load distribution and improved capacity/risk profile after.

Before (Bad State) Test Design

- Run only app on :8080 (no Nginx, single backend).
- Execute mixed traffic load.

Expected bad outcomes: - Single-node saturation risk. - No failover path. - Lower horizontal scalability.

After (Good State) Test Design

- Run app-1, app-2, nginx under lb profile.
- Run same load via :8088.

Expected good outcomes: - Requests observed on both upstreams. - Higher resilience and better connection balancing.

Suggested Execution Commands

- `docker compose --profile lb up -d --build app-1 app-2 nginx`
- `for i in {1..60}; do curl -s -D - http://localhost:8088/api/products -o /dev/null | rg "X-Upstream-Addr"; done`
- `./scripts/verify_phase5.sh t3`

Metrics to Capture

- Upstream request distribution ratio.
 - Throughput difference single vs multi instance.
 - Error rate under burst load.
-

6) Distributed Caching

Requirement Meaning

Use distributed cache (Redis) for hot products and reduce DB pressure.

What In This Project Implements It

- Cache manager and TTL:
 - `src/main/java/com/parallelcart/config/CacheConfig.java`
- Cached product reads:
 - `src/main/java/com/parallelcart/service/impl/ProductServiceImpl.java`
- Post-commit invalidation:
 - `CacheInvalidationServiceImpl`
 - Triggered from checkout in `CartServiceImpl`.
- Runtime config:
 - `application-local.yml` (`app.cache.product.ttl-seconds`)
- Test coverage:
 - `ProductServiceCachingTest`

Deep Technical Explanation

`listProducts` and `getProduct` are annotated with `@Cacheable` using deterministic keys (`all, id:<id>`). Redis cache manager applies product TTL. On inventory mutation from checkout, invalidation runs after transaction commit, not before, preventing stale cache inconsistencies if checkout later rolls back.

This design lowers repeated read latency and DB load while preserving correctness after writes.

Concrete Project Evidence

- Cache key strategy in `ProductServiceImpl`.
- Redis + TTL config in `CacheConfig`.
- Transaction synchronization hook (`afterCommit`) in `CacheInvalidationServiceImpl`.
- Verification script `verify_phase4.sh` tasks `t1` and `t2`.

Before/After Test Plan (Point 6)

Goal

Show reduction in read latency and DB query pressure after Redis cache.

Before (Bad State) Test Design

- Start app with `SPRING_CACHE_TYPE=none`.
- Run read-heavy k6 scenario.

Expected bad outcomes: - Higher average/p95 latency. - Higher direct DB query volume.

After (Good State) Test Design

- Start app with `SPRING_CACHE_TYPE=redis`.
- Run same exact k6 profile.

Expected good outcomes: - Lower avg/p95 latency (for hot reads). - Better throughput and reduced DB pressure. - Cache keys visible in Redis and correctly invalidated after checkout.

Suggested Execution Commands

- `TARGET_VUS=220 HOLD=120s ./scripts/benchmark_phase4.sh both`
- `./scripts/verify_phase4.sh t1`
- `./scripts/verify_phase4.sh t2`

Metrics to Capture

- Avg and p95 latency before/after.
 - Throughput before/after.
 - Cache hit/miss proxy via repository call counts or DB metrics.
 - Redis key existence around mutation events.
-

7) Concurrency Control (Optimistic/Pessimistic Locking)

Requirement Meaning

Explicitly implement optimistic or pessimistic locking for sensitive stock updates.

What In This Project Implements It

- Optimistic lock: `Inventory.@Version`.
- Pessimistic lock: `InventoryRepository.findByProductForUpdate` with `LockModeType.PESSIMISTIC_WRITE`.
- Strategy orchestration: `CartServiceImpl` retry loop then fallback lock.

Deep Technical Explanation

Requirement #7 is the formal locking strategy requirement, while requirement #1 is broader integrity goal. This project fulfills both by combining mechanisms: - optimistic path for speed under normal concurrency. - pessimistic fallback for correctness under high contention.

This hybrid method avoids permanent pessimistic locking cost while guaranteeing a hard serialization escape hatch.

Concrete Project Evidence

- Inventory lock annotations and repository method definitions.
- Retry-and-fallback flow in checkout service.

Before/After Test Plan (Point 7)

Goal

Demonstrate lock strategy effect under high stock contention.

Before (Bad State) Test Design

- Experimental branch with only naive updates (no version lock, no FOR UPDATE).
- High contention on one SKU.

Expected bad outcomes: - Data races and occasional inconsistent final stock. - Non-deterministic outcomes.

After (Good State) Test Design

- Restore current optimistic+fallback design.
- Same contention profile.

Expected good outcomes: - Deterministic final stock. - Conflict handled by retry/fallback, not corruption.

Suggested Execution Commands

- Build dedicated concurrency test class (recommended new test):
InventoryContentionConcurrencyTest.
- Existing functional guardrail:
 - `./mvnw -q -Dtest=CheckoutTransactionIntegrityTest test`

Metrics to Capture

- Conflict count and retry depth.
 - Fallback lock usage frequency.
 - Inventory consistency across runs.
-

8) Transaction Integrity / ACID

Requirement Meaning

Composite operation (payment + inventory update + order create) must either fully commit or fully roll back.

What In This Project Implements It

- `@Transactional` checkout boundary in `CartServiceImpl.checkout(...)`.
- Single flow creates order, payment, inventory reservation, cart cleanup, outbox event write.
- Unique idempotency key on order (`orders.idempotency_key`) to prevent duplicate logical transaction.
- Integration test coverage in `CheckoutTransactionIntegrityTest`.

Deep Technical Explanation

All checkout mutation steps happen inside one transaction. If any step throws runtime exception (for example invalid total, insufficient inventory), Spring rolls back all DB mutations in same transaction scope. Because outbox event is written inside same transaction, event consistency aligns with DB commit.

Idempotency adds safety at request layer: duplicate checkout calls with same key return same order/payment instead of creating duplicates.

Concrete Project Evidence

- `@Transactional` and checkout flow in `CartServiceImpl`.
- Unique column constraint on `Order.idempotencyKey`.
- Rollback and idempotency test cases in `CheckoutTransactionIntegrityTest`.

Before/After Test Plan (Point 8)

Goal

Show partial-state corruption before transaction boundary and all-or-nothing behavior after.

Before (Bad State) Test Design

- Experimental branch removing transactional boundary or splitting writes across non-transactional methods.

- Inject controlled failure after inventory decrement but before payment/order completion.

Expected bad outcomes: - Partial persisted state (e.g., inventory reduced but no order/payment).

After (Good State) Test Design

- Restore transactional checkout.
- Same failure injection point.

Expected good outcomes: - No partial writes remain. - Inventory/order/payment states roll back consistently.

Suggested Execution Commands

- `./mvnw -q -Dtest=CheckoutTransactionIntegrityTest test`
- Manual duplicate-idempotency request flow from docs/PHASE_VERIFICATION.md (P2 section).

Metrics to Capture

- Count of orphan rows after failure.
 - Inventory delta after failure scenario.
 - Duplicate request deduplication rate.
-

9) Stress Testing (>=100 concurrent users)

Requirement Meaning

Provide proof system can serve at least 100 concurrent users without crash/data loss.

What In This Project Implements It

- Stress script: `scripts/stress_p6.sh`
- k6 scenario: `scripts/k6_products_read.js`
- Artifacts:
 - `reports/stress/p6-stress_20260518_211019.json`
 - `reports/stress/p6-stress_20260518_211019.log`
 - `reports/stress/p6-stress_20260518_211019.md`

Deep Technical Explanation

Stress profile used: - Target virtual users: 120 (exceeds minimum 100) - Ramp up: 20s - Hold: 45s - Ramp down: 20s

Measured results from artifact: - Avg latency: 2.21 ms - P95 latency: 5.41 ms - Throughput: 7482.53 req/s - Failed request rate: 0.000000

This satisfies core requirement for concurrent stability in tested workload.

Concrete Project Evidence

- Script automatically starts stack, runs k6, exports JSON summary, parses KPI metrics.
- Log shows scenario execution across full VU timeline.

Before/After Test Plan (Point 9)

Goal

Show unstable behavior before performance/concurrency controls and stable behavior after.

Before (Bad State) Test Design

- Run stress on baseline branch with selected protections reduced (no cache, no saturation guard, no optimized concurrency handling).
- Keep same VU/time profile for fair comparison.

Expected bad outcomes: - Higher latency. - Potential non-zero failed rate. - Lower sustained throughput.

After (Good State) Test Design

- Run stress on current production branch.
- Same VU/time profile.

Expected good outcomes: - Stable run to completion. - Near-zero failures. - Better latency/throughput profile.

Suggested Execution Commands

- TARGET_VUS=120 RAMP_UP=20s HOLD=45s RAMP_DOWN=20s
./scripts/stress_p6.sh

Metrics to Capture

- http_req_failed rate.
- avg/p90/p95/max latency.

- requests per second.
 - app health and resource utilization snapshots.
-

10) Benchmarking & Bottleneck Analysis

Requirement Meaning

Measure critical operation latency, identify at least one bottleneck, and present numeric before/after comparison.

What In This Project Implements It

- AOP timing instrumentation:
 - `src/main/java/com/parallelcart/observability/PerformanceTimingAspect.java`
- Metrics script:
 - `scripts/metrics_snapshot_p6.sh`
- Bottleneck report:
 - `reports/benchmark/p6-bottleneck-analysis.md`
- Before/after stress artifacts:
 - baseline: `reports/stress/p6-stress_20260518_211019.json`
 - optimized: `reports/stress/p6-stress-opt_20260518_211441.json`

Deep Technical Explanation

Team identified candidate bottleneck in product-read entity materialization/mapping path. Optimization attempted: repository-level projection returning `ProductResponse` directly.

Measured outcome (same profile): - Avg latency: 2.21 -> 2.52 ms (worse) - P95 latency: 5.41 -> 6.46 ms (worse) - Throughput: 7482.53 -> 7295.13 req/s (worse)

This still satisfies requirement #10 because requirement asks for measured analysis and before/after comparison, not guaranteed improvement. Engineering value exists because hypothesis was tested and results were documented numerically.

Concrete Project Evidence

- AOP logs `perf_timing operation=... duration_ms=...` for product reads, checkout, inventory updates.

- Benchmark report includes bottleneck hypothesis, code-change scope, and computed delta percentages.

Before/After Test Plan (Point 10)

Goal

Maintain repeatable benchmarking framework and evolve bottleneck fixes based on data.

Before (Bad State) Test Design

- Baseline run with current stable path.
- Capture latency/throughput and AOP timing distributions.

After (Good State) Test Design

- Apply one targeted optimization per experiment (for example checkout lock/flush path).
- Re-run exact same profile and compare.

Expected good outcomes (target for next iteration): - Lower p95 checkout latency.
- Equal or higher throughput. - No correctness regressions.

Suggested Execution Commands

- `./scripts/metrics_snapshot_p6.sh`
- `TARGET_VUS=120 RAMP_UP=20s HOLD=45s RAMP_DOWN=20s
PROFILE_NAME=p6-stress ./scripts/stress_p6.sh`
- Repeat after optimization with `PROFILE_NAME=p6-stress-opt`
- Compare with `reports/benchmark/p6-bottleneck-analysis.md` method.

Metrics to Capture

- operation-level timing (`perf_timing` logs).
- global HTTP latency quantiles.
- throughput delta.
- error-rate delta.

Consolidated Before/After Validation Matrix (All 10 Points)

Point	Baseline (Bad / Control)	Improved (Good / Current)	Primary Success Signal
1	remove race protections	optimistic+retry+ pessimistic	no oversell/inventory

Point	Baseline (Bad / Control)	Improved (Good / Current)	Primary Success Signal
			corruption
2	no guard / weak bounds	bounded pools + semaphore + 503 fail-fast	controlled overload behavior
3	inline side effects	async outbox + Kafka consumers	lower checkout latency, reliable eventual side effects
4	monolithic aggregation	chunked + checkpoint resume	stable memory and resumable progress
5	single app instance	nginx + app-1/app-2 least_conn	requests distributed to multiple upstreams
6	cache disabled	redis cache + post-commit invalidation	lower read latency, consistent invalidation
7	naive locking	optimistic then pessimistic fallback	deterministic stock under contention
8	no transaction boundary	transactional checkout + idempotency	all-or-nothing writes, no duplicates
9	reduced protections baseline	full current setup	>=100 concurrent users with stability
10	unoptimized baseline	candidate optimization and re-benchmark	numeric before/after with reproducible method

Practical Execution Plan for Full Demonstration

Phase A: Build Two Comparable Targets

1. baseline-control branch (feature toggles reduced per point).
2. current-optimized branch (existing implementation).

Phase B: Standardize Dataset and Environment

1. Reset environment before each scenario: `docker compose down -v`.
2. Start with identical configs except feature under test.
3. Use identical user/product dataset for fair comparison.

Phase C: Run Point-by-Point Protocol

1. Execute each point's baseline test.
2. Capture metrics and store artifacts in timestamped files.
3. Execute improved test with same profile.
4. Compare and summarize numerical deltas.

Phase D: Report Artifact Structure

Recommended folder structure for final demonstration run: - reports/before-after/p1/... - reports/before-after/p2/... - ... - reports/before-after/p10/...

Each point folder should include: - command log - KPI JSON/CSV - brief markdown interpretation - pass/fail decision against threshold

Gaps and Recommended Additions

1. Add a dedicated high-contention integration test for race-condition proof (@RepeatedTest + parallel checkout threads on same SKU).
 2. Add automated KPI threshold checks in CI (fail build if p95 or error-rate regresses beyond tolerated limits).
 3. Add focused checkout contention stress profile (current stress is read-heavy products workload).
 4. Add dashboards (Prometheus/Grafana) to retain time-series evidence beyond one-off logs.
-

Final Assessment

The project aligns strongly with the ten non-functional requirements in the PDF. Implementation is not only structural but also operationally verifiable through tests and scripts. The requested “before and after” methodology can be executed systematically with the included plan, and most required evidence pipelines already exist in repository scripts.